第15课 枚举法（一）

> 本讲义配合课堂例题讲解使用，代码可直接在 Dev-C++ 中运行验证。

五、课堂例题

例题 1：找出 1 到 N 之间的所有完全平方数

题目：输入一个正整数 N（N ≤ 10000），输出 1 到 N 之间所有的完全平方数，每行一个。

讲解思路：
1. 枚举什么？——枚举 1 到 N 的每一个数 i。
2. 条件是什么？——i 是完全平方数，即 i 开根号后取整再平方等于 i。
3. 用 `(int)sqrt(i)` 取整后再平方，和 i 比较。

方法一（枚举每个数去判断）：

```
#include <iostream>
#include <cmath>
using namespace std;

int main() {
    int n;
    cin >> n;
    for (int i = 1; i <= n; i++) {
        int s = (int)sqrt(i);       // 开根号并取整
        if (s * s == i) {            // 平方回去看看是不是等于 i
            cout << i << endl;
        }
    }
    return 0;
}
```

方法二（换个角度枚举，更聪明）：

> 老师启发：与其检查每个数是不是完全平方数，不如直接枚举"谁的平方在范围内"。

```
#include <iostream>
using namespace std;

int main() {
    int n;
    cin >> n;
    for (int i = 1; i * i <= n; i++) {
        cout << i * i << endl;
    }
    return 0;
}
```

> **引导提问**：方法一循环了 N 次，方法二循环了多少次？只有 √N 次！快了好多倍！这就是**缩小枚举范围**的威力。

例题 2：找出所有的水仙花数

题目：输出所有的三位水仙花数。水仙花数是指一个三位数，其各位数字的立方和等于该数本身。

讲解思路：
1. 枚举什么？——枚举所有三位数，即 100 到 999。
2. 条件是什么？——百位^3 + 十位^3 + 个位^3 == 这个数本身。
3. 如何拆位？用 `/` 和 `%`：百位 = i/100，十位 = i/10%10，个位 = i%10。

```
#include <iostream>
using namespace std;

int main() {
    for (int i = 100; i <= 999; i++) {
        int a = i / 100;        // 百位
        int b = i / 10 % 10;    // 十位
        int c = i % 10;         // 个位
        if (a*a*a + b*b*b + c*c*c == i) {
            cout << i << endl;
        }
    }
    return 0;
}
```

运行结果：

```
153
370
371
407
```

> **成就感设计**：让学生运行后数一数，一共只有 4 个水仙花数！电脑试了 900 个数，帮我们在一眨眼之间找出来了。

例题 3：百钱买百鸡（简化版）

题目：公鸡 5 元一只，母鸡 3 元一只，小鸡 1 元买 3 只。用 100 元钱买 100 只鸡，每种至少 1 只，有哪些买法？输出所有方案（公鸡数 母鸡数 小鸡数）。

讲解思路：
1. 枚举什么？——枚举公鸡数量 x 和母鸡数量 y。
2. 小鸡数量 z = 100 - x - y，不用枚举，直接算。
3. 条件是什么？——总价 = 5x + 3y + z/3 == 100，且 z > 0 且 z 能被 3 整除。
4. 枚举范围：x 从 1 到 19（至少 1 只，至多 100÷5=20，留余地给其他鸡），y 从 1 到 32。

```
#include <iostream>
using namespace std;

int main() {
    int count = 0;  // 记录方案数
    for (int x = 1; x <= 19; x++) {            // 枚举公鸡数量
        for (int y = 1; y <= 32; y++) {         // 枚举母鸡数量
            int z = 100 - x - y;                // 小鸡数量直接算
            if (z >= 1 && z % 3 == 0) {         // 小鸡至少1只，且能3只3只地买
                int cost = 5 * x + 3 * y + z / 3;  // 算总花费
                if (cost == 100) {
                    cout << "公鸡:" << x << " 母鸡:" << y << " 小鸡:" << z << endl;
                    count++;
                }
            }
        }
    }
    cout << "共有 " << count << " 种买法" << endl;
    return 0;
}
```

运行结果：

```
公鸡:4 母鸡:18 小鸡:78
公鸡:8 母鸡:11 小鸡:81
公鸡:12 母鸡:4 小鸡:84
共有 3 种买法
```

> **教学重点回顾**：这道题我们只用了两重循环而不是三重循环，因为知道了公鸡和母鸡的数量，小鸡数量可以直接算出来。**能算就不枚举**——这就是优化！

附：例题源代码与运行结果

例题 1：Find All Perfect Squares from 1 to N

```
// Input a positive integer N (<= 10000), output all perfect squares
// from 1 to N, one per line.
// Method 1: Check each number from 1 to N.
#include <iostream>
#include <cmath>
using namespace std;

int main() {
    int n;
    cin >> n;
    for (int i = 1; i <= n; i++) {
        int s = (int)sqrt(i);       // integer square root
        if (s * s == i) {            // square it back to verify
            cout << i << endl;
        }
    }
    return 0;
}
```

Sample Input

```
30
```

Expected Output

```
1
4
9
16
25
```

例题 2：Find All Narcissistic Numbers

```
// A narcissistic number is a 3-digit number where the sum of cubes
// of its digits equals the number itself. Output all such numbers.
#include <iostream>
using namespace std;

int main() {
    for (int i = 100; i <= 999; i++) {
        int a = i / 100;        // hundreds digit
        int b = i / 10 % 10;    // tens digit
        int c = i % 10;         // ones digit
        if (a * a * a + b * b * b + c * c * c == i) {
            cout << i << endl;
        }
    }
    return 0;
}
```

Expected Output

```
153
370
371
407
```

例题 3：Hundred Coins for Hundred Chickens

```
// Rooster costs 5 yuan, hen costs 3 yuan, 3 chicks cost 1 yuan.
// Buy exactly 100 chickens with exactly 100 yuan, at least 1 of each.
// Output all possible combinations.
#include <iostream>
using namespace std;

int main() {
    int count = 0;  // count number of solutions
    for (int x = 1; x <= 19; x++) {           // enumerate roosters
        for (int y = 1; y <= 32; y++) {        // enumerate hens
            int z = 100 - x - y;               // chicks calculated directly
            if (z >= 1 && z % 3 == 0) {        // at least 1 chick, must be multiple of 
                int cost = 5 * x + 3 * y + z / 3;  // total cost
                if (cost == 100) {
                    cout << "Rooster:" << x
                         << " Hen:" << y
                         << " Chick:" << z << endl;
                    count++;
                }
            }
        }
    }
    cout << "Total " << count << " solutions" << endl;
    return 0;
}
```

Expected Output

```
Rooster:4 Hen:18 Chick:78
Rooster:8 Hen:11 Chick:81
Rooster:12 Hen:4 Chick:84
Total 3 solutions
```